claude-windows / 9f6357d8-0e35-491a-a830-052e094c7f37

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\9f6357d8-0e35-491a-a830-052e094c7f37\subagents\workflows\wf_e3cc36d8-eca\agent-a37f304f56c1ba26d.jsonl`
- SHA-256: `9a76edf0fd7e7242e20984cc838226d1cd197cae786f6d3b2b060f25b42dd633`
- Source modified: `2026-06-21T09:18:33+00:00`
- Imported at: `2026-07-05T16:48:25+00:00`
- project: `wf_e3cc36d8-eca`
- session_id: `9f6357d8-0e35-491a-a830-052e094c7f37`

Transcript

1. user (2026-06-21T09:17:04.589Z)

You are verifying a precise claim in the engine repo at C:/Users/avidu/Projects/badminton-highlight-indexer.

CLAIM: "POST /api/process already accepts a gs:// (or gcs://) video_path, fetches it, runs the full pipeline, and writes a queryable video_id into the API's SQLite DB (videos/play_segments) ? so the SPA can load it via ?video=<id>." This is under DEFAULT single-user config (no security.allowed_video_root set; storage.input_backend defaults to local).

Trace the WHOLE chain with file:line evidence and decide if the claim holds. Read at least:
- backend/main.py (the @app.post("/api/process") endpoint ~line 260, and _execute_processing ~line 140-258)
- backend/storage/__init__.py (resolve_input_ref, acquire_local_input, fetch)
- backend/storage/ref.py (parse_uri, resolve_input_uri ? esp. gs->gcs normalization and explicit-scheme-wins)
- backend/utils/paths.py (validate_input_path)
- backend/api/job_worker.py (how a queued job is drained to completion and how _job_to_request rebuilds the ProcessRequest from the stored video_path)
- backend/storage/gcs.py (fetch_to_local)

Pay special attention to: the submit-time existence check (does it skip non-local backends?), validate_input_path with allowed_root=None, whether the stored job.video_path round-trips the gs:// string through the worker to acquire_local_input, and whether the DB write (add_video / play_segments) is unconditional on the fetched local file. Separate genuine CODE gates from mere runtime prerequisites (the network fetch needing google-cloud-storage + ADC auth) ? the latter are environmental, not a code gap. Return structured JSON.

2. assistant (2026-06-21T09:17:07.080Z)

I'll trace the whole chain with file:line evidence. Let me start by reading the key files in parallel.

3. user (2026-06-21T09:17:10.229Z)

Your configuration specifies to merge with the ref 'refs/heads/feat/serving-m4-device-context' from the remote, but no such ref was fetched.

4. user (2026-06-21T09:17:10.243Z)

```
1       """Pluggable storage layer: local (default) + S3/GCS first-class buckets + Google Drive.
2
3       URI-driven (see ``ref.parse_uri``). The package import is cheap ? cloud SDKs are
imported
4       lazily, only when a non-local bucket URI is actually used (install the ``storage-s3`` /
5       ``storage-gcs`` extras). The default ``local`` backend needs no third-party deps and
```

```
preserves
6       today's behaviour byte-for-byte; ``gdrive://`` likewise needs none ? it reads a locally
MOUNTED
7       Google Drive (Google Drive for Desktop) as plain files.
8
9       Worker/dispatcher entry points speak URIs via the thin helpers ``fetch`` / ``put`` /
10      ``list_uris`` / ``get_storage``; ``local://`` (and bare paths) make ``fetch`` an
identity
11      passthrough. See ``docs/DECOUPLED_COMPUTE/``.
12      """
13      from __future__ import annotations
14
15      import os
16      import tempfile
17      from typing import Any, Dict, Iterator, Optional
18
19      from backend.storage.base import StorageBackend
20      from backend.storage.ref import StorageRef, StorageStat, parse_uri, resolve_input_uri
21
22      __all__ = [
23          "StorageBackend", "StorageRef", "StorageStat", "parse_uri", "resolve_input_uri",
24          "get_storage", "fetch", "put", "list_uris",
25          "acquire_local_input", "input_is_local", "resolve_input_ref",
26      ]
27
28
29      def _storage_dict(storage_config: Any) -> Dict[str, Any]:
30          """Coerce a StorageConfig (Pydantic) / dict / None into a plain settings dict."""
31          if storage_config is None:
32              return {}
33          if hasattr(storage_config, "model_dump"):
34              return storage_config.model_dump()  # type: ignore[no-any-return]
35          return dict(storage_config)
36
37
38      def resolve_input_ref(raw: str, storage_config: Any = None) -> StorageRef:
39          """Resolve a user-supplied input (a path or storage URI) into a ``StorageRef`` under
the input
40          config (``input_backend``/``input_root``). Raises ``ValueError`` for an unsupported
URI scheme ?
41          callers at the API/CLI boundary catch that and return a clean client error (never a
500).
42
43          Use ``ref.backend`` to branch local-vs-remote and ``ref.key`` for the resolved LOCAL
path (e.g.
44          ``local://?`` is stripped to the bare path), rather than stat-ing the raw URI
string."""
45          sc = _storage_dict(storage_config)
46          uri = resolve_input_uri(raw, input_backend=sc.get("input_backend", "local"),
47                                  input_root=sc.get("input_root", ""))
48          return parse_uri(uri)
49
50
51      def input_is_local(raw: str, storage_config: Any = None) -> bool:
52          """True if ``raw`` resolves (under the input config) to a LOCAL filesystem path ?
i.e. a
53          plain on-disk file the caller can stat/validate directly. False for a bucket / Drive
URI
54          that must be fetched first. Raises ``ValueError`` for an unsupported scheme (same as
55          ``resolve_input_ref``)."""
56          return resolve_input_ref(raw, storage_config).backend == "local"
57
58
59      def acquire_local_input(raw: str, storage_config: Any = None, *,
60                              scratch_parent: "str | None" = None) -> str:
```

```
        """Resolve ``raw`` (a path or storage URI) to a LOCAL file path ready for
processing.

        Local / bare paths are returned UNCHANGED (identity passthrough, no copy ? today's
behaviour
        byte-for-byte). A bucket / Drive source (``gs://``/``s3://``/``gdrive://``, or a
bare name under a
        non-local ``input_root``/``input_backend``) is downloaded into a fresh scratch dir
and the local
        path returned. The scratch copy is left under the system temp dir (a pure
intermediate,
        regenerable from the source) and relies on OS temp reclamation ? the same pull
pattern the
        worker's ``cmd_infer`` already ships."""
        sc = _storage_dict(storage_config)
        ref = resolve_input_ref(raw, sc)
        if ref.backend == "local":
            return ref.key  # identity ? no temp, no copy, byte-identical to passing the
path directly
        scratch = tempfile.mkdtemp(prefix="rally-input-", dir=scratch_parent)
        return fetch(ref.uri, os.path.join(scratch, ref.name or "in.mp4"), config=sc)


    def _backend_class(name: str):
        """Lazy-resolve a backend class so cloud SDKs aren't imported until needed."""
        if name == "local":
            from backend.storage.local import LocalStorage
            return LocalStorage
        if name == "s3":
            from backend.storage.s3 import S3Storage
            return S3Storage
        if name == "gcs":
            from backend.storage.gcs import GCSStorage
            return GCSStorage
        if name == "gdrive":
            from backend.storage.gdrive import GDriveStorage
            return GDriveStorage
        raise ValueError(f"unknown storage backend: {name!r} (expected
local|s3|gcs|gdrive)")


    def get_storage(backend: Optional[str] = None, *,
                    config: Optional[Dict[str, Any]] = None, **overrides) -> StorageBackend:
        """Construct a backend. ``backend`` (or ``config['backend']``, default 'local')
selects
        the implementation; its per-backend settings come from ``config[backend]`` merged
with
        ``overrides``. Settings carry NO secrets ? only endpoints/regions/paths/source-
selectors."""
        cfg = config or {}
        name = backend or cfg.get("backend", "local")
        settings: Dict[str, Any] = dict(cfg.get(name, {}) or {})
        settings.update(overrides)
        return _backend_class(name)(**settings)


    def fetch(uri: str, dst: "str | os.PathLike[str] | None" = None, *,
              config: Optional[Dict[str, Any]] = None, overwrite: bool = False) -> str:
        """Resolve ``uri`` to a local file path. For ``local`` this is an identity
passthrough
        (no copy); for s3/gcs it downloads to ``dst``."""
        ref = parse_uri(uri)
        return get_storage(ref.backend, config=config).fetch_to_local(ref, dst,
overwrite=overwrite)

```

```
113
114     def put(local_path: "str | os.PathLike[str]", uri: str, *,
115             config: Optional[Dict[str, Any]] = None, content_type: Optional[str] = None) ->
str:
116         """Upload/copy ``local_path`` to ``uri``; return ``uri``."""
117         ref = parse_uri(uri)
118         get_storage(ref.backend, config=config).put(local_path, ref,
content_type=content_type)
119         return uri
120
121
122     def list_uris(prefix_uri: str, *, config: Optional[Dict[str, Any]] = None) ->
Iterator[str]:
123         """Yield the URI of every object under ``prefix_uri``."""
124         ref = parse_uri(prefix_uri)
125         for r in get_storage(ref.backend, config=config).list(ref):
126             yield r.uri
127
```

5. user (2026-06-21T09:17:10.712Z)

```
1       import os
2       import sys
3       import argparse
4       import logging
5       import uuid
6       from urllib.parse import quote
7       import uvicorn
8       from dotenv import load_dotenv
9
10      load_dotenv()
11
12      # Centralized logging setup for the indexer (hot paths use logger, CLI output uses print
for direct user-facing messages).
13      logging.basicConfig(
14          level=logging.INFO,
15          format='%(asctime)s [%(levelname)s] %(name)s: %(message)s',
16          datefmt='%H:%M:%S'
17      )
18      logger = logging.getLogger(__name__)
19      from fastapi import FastAPI, HTTPException
20      from fastapi.middleware.cors import CORSMiddleware
21      from pydantic import BaseModel
22      from typing import List, Dict, Any, Optional
23
24      from fastapi.staticfiles import StaticFiles
25      from fastapi.responses import FileResponse, HTMLResponse, JSONResponse
26      from backend.pipeline.validators.base import registry
27      from backend.infrastructure.database import Database
28      from backend.pipeline.segmenters.base import segmenter_registry
29      from backend.pipeline.stitcher.compiler import VideoCompiler
30      from backend.utils.hashing import compute_video_id  # canonical file-identity hashing
31      from backend.utils.paths import resolve_under_root, safe_output_filename,
validate_input_path
32      from backend.config import load_config as load_app_config, AppConfig, StitchingConfig  #
new typed config
33      from backend.results import Failure  # standardized error/result contract
34      from backend.reporting import emit_indexer_report
35      from backend import storage  # M2: URI-aware input acquisition (local = identity
passthrough)
36
37      app = FastAPI(title="Sports Highlight Indexer API")
38
39      # Enable CORS for the local web interface. allow_origins='*' with allow_credentials=True
is
```

```
    # rejected by browsers per the fetch spec and is an unsafe default to carry into the
    # auth/tenancy work; this tool uses no cookies, so credentials stay off.
    app.add_middleware(
        CORSMiddleware,
        allow_origins=["*"],
        allow_credentials=False,
        allow_methods=["*"],
        allow_headers=["*"],
    )

    # Serves static frontend files directly (now under api/ per approved structure)
    os.makedirs("backend/api/static", exist_ok=True)
    app.mount("/static", StaticFiles(directory="backend/api/static"), name="static")

    @app.get("/", response_class=HTMLResponse)
    def serve_index():
        index_path = "backend/api/static/index.html"
        if os.path.exists(index_path):
            with open(index_path, "r", encoding="utf-8") as f:
                return f.read()
        return "<h3>Sports Highlight Indexer Server is Running. Please create index.html in
backend/api/static/</h3>"

    # Global variables initialized at startup
    db_instance: Optional[Database] = None
    global_config: Optional[AppConfig] = None  # now typed (Pydantic model)

    # Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
P3)
    # now lives in backend.api.job_worker. Re-exported here so the names stay addressable on
    # `backend.main` (`JobWaiting`, `_drain_due_jobs`, `_arm_wake_timer`, `_get_executor`,
    # `_run_claimed_job`, `_MAX_DRAIN_WAKE_SEC`, ...). The worker resolves these seams
THROUGH
    # this module object at call-time, so `monkeypatch.setattr(backend.main, ...)` still
    # intercepts the inter-function calls (a parked job arms the PATCHED `_arm_wake_timer`,
    # so no real daemon timer leaks in tests). Behavior is unchanged by the extraction.
    from backend.api.job_worker import (  # noqa: F401  (re-exports: names stay addressable
on backend.main)
        JobWaiting,
        _arm_wake_timer,
        _drain_due_jobs,
        _get_executor,
        _job_to_request,
        _MAX_DRAIN_WAKE_SEC,
        _run_claimed_job,
        _schedule_next_drain_if_waiting,
    )

    # Legacy thin wrapper for any remaining direct calls during transition.
    # New code should import load_app_config / AppConfig from backend.config directly.
    def load_config(config_path: str = "config.json") -> Dict[str, Any]:
        cfg = load_app_config(config_path)
        return cfg.model_dump(mode="json")

    # --- API Models ---
    class ProcessRequest(BaseModel):
        video_path: str
        player_pool: Optional[List[str]] = None
        max_frames: Optional[int] = None
        segmenter: Optional[str] = None

    class QueryRequest(BaseModel):
        video_id: str
        server: Optional[str] = None
        receiver: Optional[str] = None
```

```
        duration_min: Optional[float] = None
        event_type: Optional[str] = None

    class CompileRequest(BaseModel):
        video_id: str
        segment_ids: List[int]
        output_filename: str

    def _finalize_reingest(video_id: str, segments, play_wm: int, cand_wm: int) -> None:
        """Commit a (re)segmentation with destroy-AFTER-confirm semantics.

        On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
        fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark)
and
        keep the prior good results intact ? a failed/empty re-run never destroys prior
data.
        """
        if segments:
            db_instance.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
        else:
            db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)


    # --- API Endpoints ---
    @app.get("/api/config")
    def get_config():
        return global_config

    @app.get("/api/health")
    def health():
        """Cheap liveness/readiness probe (no auth) for the SPA's offline banner
        (HOSTING_PLAN.md:42). Reports the configured sport + default segmenter so the UI can
        confirm the engine is reachable and which detector is wired. Never raises."""
        sport = getattr(global_config, "sport", None)
        indexing = getattr(global_config, "indexing", None)
        return {
            "status": "ok",
            "sport": sport,
            "default_segmenter": getattr(indexing, "default_segmenter", None),
        }

    def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
        """The full hash ? validate ? segment ? report pipeline for one video.

        This is the former synchronous /api/process body, unchanged in behavior, now run on
        the job worker thread (DEFERRED_HARDENING_PLAN #16). Returns the same response dict
        the sync endpoint used to return (UI compatibility); the per-video observability
        report is still emitted in `finally:` and grafted as response["reports"].

        `progress` is an optional callable(stage: str) used to surface coarse job progress.
        Raises on unexpected internal errors ? the caller records those as a job failure
        (after this function has already restored the pre-run segments, see below).
        """
        notify = progress or (lambda stage: None)

        notify("hashing")
        # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
(identity ?
        # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
original
        # req.video_path (the source URI) is kept for the DB record + report; only the file-
reading
        # consumers (hash / validators / segmenter) use the resolved local path.
        local_video = storage.acquire_local_input(req.video_path, getattr(global_config,
"storage", None))
```

```
        video_id = compute_video_id(local_video)
        was_reingest = db_instance.get_video(video_id) is not None

        # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
report)
        notify("validating")
        logger.info(f"Running validations for {req.video_path}...")
        val_results, val_context = registry.run_all_with_context(local_video, global_config)
        failures = [r for r in val_results if not r.passed]

        # typed AppConfig: attribute access for the default segmenter (with safe fallback)
        default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
"motion")
        seg_name = req.segmenter or default_seg
        segmenter_actual = None
        segmentation_ran = False
        response: Optional[Dict[str, Any]] = None
        try:
            if failures:
                # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
                # status; it no longer destroys the video's prior good segments.
                db_instance.add_video(video_id, req.video_path, "failed", [r.model_dump()
for r in val_results])
                response = {
                    "video_id": video_id,
                    "status": "failed",
                    "message": "Video failed validation checks.",
                    "results": [r.model_dump() for r in val_results],
                }
                return response

            # 2. Run segmenter & indexer
            logger.info("[API] Video passed checks. Segmenting and indexing...")
            db_instance.add_video(video_id, req.video_path, "processed", [r.model_dump() for
r in val_results])

            segmenter_cls = segmenter_registry.get(seg_name)
            if not segmenter_cls:
                # Submit-time validation already 400s unknown names; this is the defensive
                # in-worker path (e.g. config default pointing at an unregistered
segmenter).
                available = list(segmenter_registry.list_available().keys())
                response = {
                    "video_id": video_id,
                    "status": "failed",
                    "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
                    "results": [r.model_dump() for r in val_results],
                    "play_segments": [],
                }
                return response

            logger.info(f"[API] Using segmenter: {seg_name}")
            notify(f"segmenting ({seg_name})")
            segmenter_actual = seg_name
            # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
            # if the run yields segments; otherwise drop the new partial rows and keep the
old.
            play_wm, cand_wm = db_instance.segment_watermarks(video_id)
            segmenter = segmenter_cls(db_instance, global_config)
            try:
                result = segmenter.process_video(video_id, local_video, req.player_pool,
req.max_frames)
            except Exception:
```

```
                # A raising segmenter must not leave half-written rows: restore the pre-run
                # state (same destroy-after-confirm contract as the Failure branch), then
let
                # the job worker record the failure.
                db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
                raise
            segmentation_ran = True

            if isinstance(result, Failure):
                logger.error(f"[API] Segmenter failed: {result.message}")
                db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
                response = {
                    "video_id": video_id,
                    "status": "failed",
                    "message": f"Segmenter '{seg_name}' failed: {result.message}",
                    "results": [r.model_dump() for r in val_results],
                    "play_segments": [],
                }
                return response

            segments = result.value if hasattr(result, "value") else result
            notify("finalizing")
            _finalize_reingest(video_id, segments, play_wm, cand_wm)

            response = {
                "video_id": video_id,
                "status": "success",
                "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.",
                "results": [r.model_dump() for r in val_results],
                "play_segments": segments,
            }
            return response
        finally:
            # Always emit the per-video observability report (next to the video, or to
            # report.output_dir). Never raises. Surface the paths in the response.
            paths = emit_indexer_report(
                db=db_instance, video_id=video_id, video_path=req.video_path,
config=global_config,
                val_results=val_results, val_context=val_context,
                segmenter_requested=req.segmenter, segmenter_actual=segmenter_actual,
                was_reingest=was_reingest, segmentation_ran=segmentation_ran,
            )
            if paths and isinstance(response, dict):
                response["reports"] = paths


    @app.post("/api/process", status_code=202)
    def process_video(req: ProcessRequest):
        """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.

        Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
        client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
        runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
        worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
        """
        allowed_root = getattr(getattr(global_config, "security", None),
"allowed_video_root", None)
        try:
            validate_input_path(req.video_path, allowed_root=allowed_root)
            # M2: resolve the input once (path or storage URI) ? raises ValueError for an
unsupported
            # scheme, which we map to a clean 400 (an unknown scheme must never 500).
            input_ref = storage.resolve_input_ref(req.video_path, getattr(global_config,
"storage", None))
```

```
        except ValueError as e:
            raise HTTPException(status_code=400, detail=str(e)) from e
        # The local-existence fast-fail applies only to a LOCAL input, and stats the
RESOLVED local path
        # (input_ref.key ? e.g. local://? stripped to the bare path), not the raw URI
string. A
        # bucket/Drive URI can't be cheaply os.path.exists()'d ? its existence is verified
when the worker
        # fetches it (a fetch miss fails the job loudly). validate_input_path above still
runs for every
        # input: its null-byte guard always applies, and a configured allowed_video_root
(hosted mode)
        # rejects a non-local URI as out-of-root.
        if input_ref.backend == "local" and not os.path.exists(input_ref.key):
            raise HTTPException(status_code=404, detail=f"Video file not found at
'{req.video_path}'")
        if req.segmenter and not segmenter_registry.get(req.segmenter):
            available = list(segmenter_registry.list_available().keys())
            raise HTTPException(
                status_code=400,
                detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
{available}"
            )

        job_id = uuid.uuid4().hex
        if not db_instance.create_job(job_id, req.video_path, req.segmenter,
                                      req.player_pool, req.max_frames):
            raise HTTPException(status_code=500, detail="Failed to persist job record.")
        _get_executor().submit(_drain_due_jobs)
        return JSONResponse(
            status_code=202,
            content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
        )


    @app.get("/api/jobs/{job_id}")
    def get_job(job_id: str):
        """Job state: {status, progress, result, reports}. `status` = execution state
        (queued|running|done|failed); the pipeline verdict is result["status"]."""
        job = db_instance.get_job(job_id)
        if not job:
            raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
        result = job.get("result")
        return {
            "job_id": job["id"],
            "status": job["status"],
            "progress": job.get("progress"),
            "video_id": job.get("video_id"),
            "error": job.get("error"),
            "result": result,
            "reports": (result or {}).get("reports"),
            "created_at": job.get("created_at"),
            "started_at": job.get("started_at"),
            "finished_at": job.get("finished_at"),
        }

    # --- Chunked upload (B2, PR-2) ------------------------------------------------------
    # The chunked-upload island (in-process registry + helpers + the 4 endpoints) now lives
in
    # backend.api.uploads as a self-contained APIRouter. It resolves the live startup config
via
    # backend.main.global_config, so behavior ? route paths, status codes, body-size limits,
    # sequential-part logic, and abort cleanup ? is unchanged.
    from backend.api import uploads as uploads_api

```

```
    app.include_router(uploads_api.router)


    # --- Highlight download (B3, PR-2) ----------------------------------------------------

    @app.get("/api/highlights/{video_id}/{filename}")
    def download_highlight(video_id: str, filename: str):
        """Stream a compiled highlight reel ? `filename` is the bare name given to
/api/compile
        (which only writes into output_dir; the browser previously got back an unreachable
        server-local path)."""
        try:
            name = safe_output_filename(filename)
        except ValueError as e:
            raise HTTPException(status_code=400, detail=str(e)) from e
        if not db_instance.get_video(video_id):
            raise HTTPException(status_code=404, detail="Indexed video record not found.")
        output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
or "output"
        path = os.path.join(output_dir, name)
        try:
            path = resolve_under_root(path, output_dir)
        except ValueError as e:
            raise HTTPException(status_code=400, detail=str(e)) from e
        if not os.path.isfile(path):
            raise HTTPException(status_code=404,
                                detail=f"No compiled file named '{name}'. Compile first via
/api/compile.")
        media = "video/quicktime" if name.lower().endswith(".mov") else "video/mp4"
        return FileResponse(path, media_type=media, filename=name)


    @app.post("/api/query")
    def query_play_segments(req: QueryRequest):
        filters = {
            "server": req.server,
            "receiver": req.receiver,
            "duration_min": req.duration_min,
            "event_type": req.event_type
        }
        segments = db_instance.query_play_segments(req.video_id, filters)
        return {"play_segments": segments}

    @app.post("/api/compile")
    def compile_highlights(req: CompileRequest):
        video = db_instance.get_video(req.video_id)
        if not video:
            raise HTTPException(status_code=404, detail="Indexed video record not found.")

        # Retrieve matching play segments from db
        all_segments = db_instance.query_play_segments(req.video_id, {})
        matched_segments = [s for s in all_segments if s["id"] in req.segment_ids]

        if not matched_segments:
            raise HTTPException(status_code=400, detail="No matching play segments found to
stitch.")

        # Reject path traversal / absolute output names (os.path.join would otherwise let an
        # absolute or '..'-laden output_filename write anywhere on disk).
        try:
            out_name = safe_output_filename(req.output_filename)
        except ValueError as e:
            raise HTTPException(status_code=400, detail=str(e)) from e

        # The configured shot-count floor (stitching.min_shot_count) applies on the API
```

```
        # path too ? parity with scripts/run_process_and_stitch.py. Segments WITHOUT
shot_count
        # metadata are exempt: the floor filters known counts only, so explicitly
        # requested manual/legacy segments can't silently vanish under the default floor.
        stitching = getattr(global_config, "stitching", None) or StitchingConfig()
        kept = []
        for s in matched_segments:
            meta = s.get("metadata") or {}
            sc = meta.get("shot_count") if isinstance(meta, dict) else None
            try:
                if sc is not None and int(sc) < stitching.min_shot_count:
                    continue
            except (TypeError, ValueError):
                pass  # unparseable count -> treat as unknown, keep
            kept.append(s)
        if len(kept) < len(matched_segments):
            logger.info(f"[compile] stitching.min_shot_count={stitching.min_shot_count}:
dropped "
                        f"{len(matched_segments) - len(kept)}/{len(matched_segments)}
segments below the floor.")
        if not kept:
            raise HTTPException(status_code=400,
                                detail=f"All {len(matched_segments)} matching segments fall
below "
f"stitching.min_shot_count={stitching.min_shot_count}.")

        # Extract start/end time pairs
        time_pairs = [(s["start_time"], s["end_time"]) for s in kept]

        # Run compiler with the configured stitching paddings + optional branding watermark
        output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
or "output"
        compiler = VideoCompiler(output_dir, begin_padding=stitching.begin_padding,
                                 end_padding=stitching.end_padding,
watermark=stitching.watermark)
        try:
            out_path = compiler.compile_highlights(video["filepath"], time_pairs, out_name)
            # `filepath` is server-local (not reachable by the browser); also return the
ready-to-use
            # download URL so the SPA needn't hand-build it (P6 / PER_RALLY_SELECTION.md
§4.3).
            download_url = f"/api/highlights/{quote(req.video_id)}/{quote(out_name)}"
            return {"status": "success", "filepath": out_path, "download_url": download_url}
        except Exception as e:
            raise HTTPException(status_code=500, detail=f"Highlight stitching failed:
{str(e)}") from e

    # --- CLI Debug Utility & Orchestration ---
    def run_cli_diagnose_clip(video_path: str, timestamp: float):
        """CLI debugging utility to examine segment properties around a timestamp."""
        print("=" * 60)
        print(f"DEBUGGING VIDEO CLIP: {video_path} at {timestamp}s")
        print("=" * 60)

        cfg = load_config()  # legacy wrapper still works, returns dict for now

        # 1. Validation check diagnostics
        print("[DIAGNOSTIC] Running validation framework...")
        results = registry.run_all(video_path, cfg)
        for r in results:
            status_symbol = "[PASS]" if r.passed else "[FAIL]"
            print(f"  {status_symbol} {r.validator_name}: {r.message}")
            if r.details:
                print(f"         Details: {r.details}")
```

```
447
448         # 2. Local segment analysis around the timestamp (sampling +/- 5 seconds)
449         print("-" * 60)
450         print("[DIAGNOSTIC] Analyzing local motion around the timestamp...")
451         import cv2
452         cap = cv2.VideoCapture(video_path)
453         if not cap.isOpened():
454             print("[FAIL] OpenCV could not open video.")
455             return
456
457         fps = cap.get(cv2.CAP_PROP_FPS)
458         total_frames = cap.get(cv2.CAP_PROP_FRAME_COUNT)
459
460         start_frame = max(0, int((timestamp - 3.0) * fps))
461         end_frame = min(int(total_frames), int((timestamp + 3.0) * fps))
462
463         # Measure frame-by-frame changes in sample region
464         cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
465         prev_gray = None
466         motions = []
467
468         for _ in range(start_frame, end_frame):
469             ret, frame = cap.read()
470             if not ret:
471                 break
472             gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
473             gray = cv2.GaussianBlur(gray, (21, 21), 0)
474
475             if prev_gray is not None:
476                 diff = cv2.absdiff(prev_gray, gray)
477                 _, thresh = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)
478                 motion_ratio = cv2.countNonZero(thresh) / (thresh.shape[0] *
thresh.shape[1])
479                 motions.append(motion_ratio)
480             prev_gray = gray
481
482         cap.release()
483
484         if motions:
485             avg_motion = sum(motions) / len(motions)
486             # Support both legacy dict (from wrapper) and future direct model
487             if hasattr(cfg, "indexing"):
488                 threshold = getattr(cfg.indexing, "motion_threshold", 0.08)
489             else:
490                 threshold = cfg.get("indexing", {}).get("motion_threshold", 0.08)  # type:
ignore[attr-defined]
491             print(f"  Sample Frame Count: {len(motions)}")
492             print(f"  Average motion index around timestamp: {round(avg_motion, 4)}
(threshold: {threshold})")
493             if avg_motion > threshold:
494                 print("  Result: [ACTIVE PLAY] High motion detected. Segment classifies as
active play.")
495             else:
496                 print("  Result: [DEAD TIME] Idle movement. Segment classified as dead
time.")
497         else:
498             print("  [ERROR] No visual frames were extracted in the sample range.")
499
500         print("=" * 60)
501
502     def main():
503         parser = argparse.ArgumentParser(description="Sports Highlight Indexer
Orchestrator")
504         parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
505         parser.add_argument("--output-dir", default="output", help="Directory where
```

```
processed clips/highlights are saved")
506         parser.add_argument("--setup", action="store_true", help="Launch dependency checker
and setup utility")
507         parser.add_argument("--debug-clip", type=float, help="Timestamp (in seconds) of a
video clip to run detailed diagnostics on")
508         parser.add_argument("--server", action="store_true", help="Run the FastAPI local API
server")
509         parser.add_argument("--port", type=int, default=8000, help="Port to run the FastAPI
server on")
510         parser.add_argument("--max-frames", type=int, help="Limit processing to the first N
sub-sampled frames for debugging")
511
512         available_segmenters = list(segmenter_registry.list_available().keys())
513         parser.add_argument("--segmenter", choices=available_segmenters, help=f"Choose rally
detection segmenter. Available: {available_segmenters}")
514         parser.add_argument("--trim-start", type=float, help="Start time in seconds for the
debug trim segment feature")
515         parser.add_argument("--trim-end", type=float, help="End time in seconds for the
debug trim segment feature")
516         parser.add_argument("--trim-output", help="Output filename for the debug trimmed
segment (saves in output-dir unless absolute path)")
517
518         args = parser.parse_args()
519
520         if args.setup:
521             # Invoke the diagnostics/bootstrap script (renamed from the old root
522             # setup.py to scripts/doctor.py so it no longer shadows the PEP 517
523             # build system). Resolve its path from this file so `indexer --setup`
524             # works regardless of the caller's cwd.
525             import subprocess
526             doctor_path = os.path.join(
527                 os.path.dirname(os.path.dirname(os.path.abspath(__file__))),
528                 "scripts", "doctor.py",
529             )
530             print(f"[INFO] Invoking diagnostics: {doctor_path} ...")
531             subprocess.run([sys.executable, doctor_path])
532             return
533
534         if args.trim_start is not None and args.trim_end is not None:
535             if not args.video_path:
536                 print("[ERROR] Please provide --video-path to extract a segment.")
537                 return
538
539             # Determine output path
540             out_filename = args.trim_output or "trimmed_segment.mp4"
541             if os.path.isabs(out_filename):
542                 out_path = out_filename
543                 out_dir = os.path.dirname(out_path)
544                 out_name = os.path.basename(out_path)
545             else:
546                 out_dir = args.output_dir
547                 out_path = os.path.join(out_dir, out_filename)
548                 out_name = out_filename
549
550             os.makedirs(out_dir, exist_ok=True)
551             print(f"[CLI] Extracting debug segment from {args.trim_start}s to
{args.trim_end}s from {args.video_path}...")
552
553             # global_config isn't initialized this early; load the typed config so the trim
554             # clip carries the same configured stitching paddings + watermark as a real
555             # highlight (watermark default-on).
556             stitching = load_app_config().stitching
557             compiler = VideoCompiler(out_dir, begin_padding=stitching.begin_padding,
558                                      end_padding=stitching.end_padding,
watermark=stitching.watermark)
```

```
            try:
                res_path = compiler.compile_highlights(args.video_path, [(args.trim_start,
args.trim_end)], out_name)
                print(f"[SUCCESS] Trimmed segment saved to: {res_path}")
            except Exception as e:
                print(f"[ERROR] Failed to compile trimmed segment: {e}")
            return

        if args.debug_clip is not None:
            if not args.video_path:
                print("[ERROR] Please provide --video-path to debug a specific clip.")
                return
            run_cli_diagnose_clip(args.video_path, args.debug_clip)
            return

        # Initialize Global Config and DB
        global global_config, db_instance
        global_config = load_app_config()  # returns typed AppConfig

        # The CLI --output-dir overrides the configured output directory. It now lives
        # on the typed model (OutputConfig.output_dir), so every consumer ? including
        # /api/compile ? reads the same value instead of a write-only runtime hack.
        global_config.output.output_dir = args.output_dir
        os.makedirs(global_config.output.output_dir, exist_ok=True)

        db_path = os.path.join(global_config.output.output_dir,
global_config.output.db_name)
        db_instance = Database(db_path)

        # Crash recovery (#16): jobs left queued/running by a previous process are dead ?
        # the executor is in-process. Mark them failed so pollers see a terminal state.
        swept = db_instance.fail_stale_jobs()
        if swept:
            logger.warning(f"[JOBS] Swept {swept} stale job(s) from a previous run ?
failed.")

        # CLI processing mode (no web UI needed)
        if args.video_path and not args.server:
            print(f"[CLI] Processing video file: {args.video_path}")
            # M2: bare/local path = identity (today's behaviour); a bucket/Drive URI is
fetched to scratch.
            # The local-existence fast-fail only applies to a local input (stat the RESOLVED
key, not the
            # raw URI); a remote fetch fails loudly. An unsupported scheme is a clean error,
not a crash.
            storage_cfg = getattr(global_config, "storage", None)
            try:
                input_ref = storage.resolve_input_ref(args.video_path, storage_cfg)
            except ValueError as e:
                print(f"[FAIL] {e}")
                return
            if input_ref.backend == "local" and not os.path.exists(input_ref.key):
                print(f"[FAIL] Video file not found: {args.video_path}")
                return
            local_video = storage.acquire_local_input(args.video_path, storage_cfg)

            video_id = compute_video_id(local_video)
            was_reingest = db_instance.get_video(video_id) is not None

            # Validate (with-context variant surfaces ffprobe_meta for the report)
            print("[CLI] Validating...")
            val_results, val_context = registry.run_all_with_context(local_video,
global_config)
            failures = [r for r in val_results if not r.passed]

```

```
617             # Save video status
618             status = "failed" if failures else "processed"
619             db_instance.add_video(video_id, args.video_path, status, [r.model_dump() for r
in val_results])
620
621             seg_name = args.segmenter or getattr(getattr(global_config, "indexing", None),
"default_segmenter", "motion")
622             segmenter_actual = None
623             segmentation_ran = False
624             try:
625                 print("\n" + "="*40)
626                 print("VALIDATION SUMMARY")
627                 print("="*40)
628                 for r in val_results:
629                     status_symbol = "[PASS]" if r.passed else "[FAIL]"
630                     print(f"{status_symbol} {r.validator_name}: {r.message}")
631                 print("="*40 + "\n")
632
633                 if failures:
634                     print("[FAIL] Video failed checks. Indexing halted.")
635                     return
636
637                 # Index
638                 segmenter_cls = segmenter_registry.get(seg_name)
639                 if not segmenter_cls:
640                     print(f"[FAIL] Segmenter '{seg_name}' not found.")
641                     return
642
643                 print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
644                 segmenter_actual = seg_name
645                 play_wm, cand_wm = db_instance.segment_watermarks(video_id)
646                 segmenter = segmenter_cls(db_instance, global_config)
647                 result = segmenter.process_video(video_id, local_video,
max_frames=args.max_frames)
648                 segmentation_ran = True
649
650                 if isinstance(result, Failure):
651                     print(f"[FAIL] Segmenter '{seg_name}' failed: {result.message}")
652                     print("[FAIL] Indexing halted for video.")
653                     segments = []
654                 else:
655                     segments = result.value if hasattr(result, "value") else result
656                     print(f"[SUCCESS] Process complete. Indexed {len(segments)} play
segments inside database: {db_path}")
657                 # Destroy-AFTER-confirm: keep new on success, restore prior on
empty/failure.
658                 _finalize_reingest(video_id, segments, play_wm, cand_wm)
659             finally:
660                 # Always emit the per-video observability report. Never raises.
661                 paths = emit_indexer_report(
662                     db=db_instance, video_id=video_id, video_path=args.video_path,
config=global_config,
663                     val_results=val_results, val_context=val_context,
664                     segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
665                     was_reingest=was_reingest, segmentation_ran=segmentation_ran,
666                 )
667                 if paths:
668                     print(f"[REPORT] Technical report : {paths.get('technical_md')}")
669                     print(f"[REPORT] Customer summary : {paths.get('customer_md')}")
670                     print(f"[REPORT] Machine JSON     : {paths.get('json')}")
671             return
672
673         # Start Local Server Mode
674         if args.server or not args.video_path:
675             print(f"[SERVER] Starting local FastAPI server on
```

```
http://localhost:{args.port}...")
676             # Resume self-scheduled work orphaned by the restart (EXECUTION_POLICY.md P3):
the
677             # in-process wake-timer for any parked 'waiting' job died with the old process,
so
678             # kick one drain ? it reclaims due jobs and re-arms the wake-timer for the rest.
679             _get_executor().submit(_drain_due_jobs)
680             uvicorn.run(app, host="127.0.0.1", port=args.port)
681
682     if __name__ == "__main__":
683         # `python -m backend.main` runs THIS file as `__main__`, but `job_worker`/`uploads`
resolve the
684         # shared mutable state (`db_instance`, `global_config`) via `import backend.main` ?
a SEPARATE
685         # module object. If `main()` runs here it configures `__main__`'s globals, leaving
686         # `backend.main.db_instance` = None; the async-job drain then crashes silently on
687         # `None.claim_due_job()` and every submitted job hangs in 'queued' forever. Dispatch
through
688         # `backend.main` so `main()` configures the module everyone actually reads. (No
recursion:
689         # imported as `backend.main`, `__name__` != "__main__", so this block doesn't re-
fire.)
690         import backend.main as _bm
691
692         _bm.main()
693
```

6. user (2026-06-21T09:17:13.873Z)

```
1       """Storage value objects + URI parsing for the pluggable storage layer.
2
3       A ``StorageRef`` is a backend-agnostic pointer to one object. URIs:
4
5           <bare path>                 -> local   (today's behaviour; e.g. C:/v/x.mp4,
/data/x.mp4)
6           local://<path>              -> local   (local:///abs or local://rel/p)
7           s3://<bucket>/<key>         -> s3       (AWS + R2 + DO Spaces + MinIO via
endpoint_url)
8           gcs://<bucket>/<key>        -> gcs      (gs:// accepted, normalized to gcs://)
9           gdrive://<path>             -> gdrive   (path under your locally-MOUNTED Google
Drive "My Drive")
10
11      No cloud SDKs are imported here ? this module is pure stdlib so ``import
backend.storage``
12      stays cheap.
13      """
14      from __future__ import annotations
15
16      import re
17      from dataclasses import dataclass
18      from datetime import datetime
19      from typing import Optional
20
21      _SCHEME_RE = re.compile(r"^([A-Za-z][A-Za-z0-9+.\-]*)://")
22      _KNOWN = ("local", "s3", "gcs", "gdrive")
23
24
25      @dataclass(frozen=True)
26      class StorageStat:
27          """Lightweight object metadata (uniform across backends)."""
28          size: int
29          modified: Optional[datetime] = None
30          etag: Optional[str] = None
31          content_type: Optional[str] = None
32
```

```
33
34      @dataclass(frozen=True)
35      class StorageRef:
36          """A backend + a location. ``bucket`` is the S3/GCS bucket; for ``local`` and
``gdrive`` it is
37          empty and ``key`` carries the path (a filesystem path, or a path under the mounted
Drive)."""
38          backend: str
39          bucket: str
40          key: str
41          uri: str
42
43          @property
44          def name(self) -> str:
45              return self.key.rstrip("/").rsplit("/", 1)[-1]
46
47
48      def parse_uri(uri: str) -> StorageRef:
49          """Parse a storage URI (or a bare filesystem path) into a ``StorageRef``.
50
51          A string with no ``scheme://`` prefix (including Windows ``C:\\...`` paths, which
52          have no ``//``) is treated as a local filesystem path ? preserving today's
behaviour.
53          """
54          s = str(uri)
55          m = _SCHEME_RE.match(s)
56          if not m:
57              return StorageRef("local", "", s, f"local://{s}")
58          scheme = m.group(1).lower()
59          rest = s[m.end():]
60          if scheme == "gs":
61              scheme = "gcs"
62          if scheme in ("local", "gdrive"):
63              # Path-keyed, no bucket: local FS, or a path under the mounted Google Drive "My
Drive".
64              return StorageRef(scheme, "", rest, s)
65          if scheme in _KNOWN:
66              bucket, _, key = rest.partition("/")
67              return StorageRef(scheme, bucket, key, s)
68          raise ValueError(f"unsupported storage URI scheme: {scheme}:// (in {s!r})")
69
70
71      def resolve_input_uri(raw: str, *, input_backend: str = "local", input_root: str = "")
-> str:
72          """Resolve a user-supplied input reference into a storage URI
(COMPUTE_DECOUPLED_SERVING M2).
73
74          Precedence (first match wins):
75            1. An explicit ``scheme://`` on ``raw`` (``gs://``, ``s3://``, ``local://``,
``gdrive://``) ? as-is.
76            2. An ABSOLUTE local path (``/data/x.mp4``, ``C:\\?``, ``C:/?``) ? as-is (always
local).
77            3. A bare/relative name + a non-empty ``input_root`` ? joined under it
78               (``"gs://bucket/videos"`` + ``"x.mp4"`` ? ``"gs://bucket/videos/x.mp4"``).
79            4. A bare/relative name + ``input_backend != "local"`` ? prefixed (``"gcs"`` +
``"bucket/x"`` ?
80               ``"gcs://bucket/x"``).
81            5. Otherwise ? ``raw`` unchanged (a relative/bare LOCAL path ? today's behaviour).
82
83          So the default (``input_backend="local"``, ``input_root=""``) is an identity
passthrough for every
84          path used today, preserving byte-for-byte behaviour.
85          """
86          s = str(raw)
87          if _SCHEME_RE.match(s):           # 1. explicit scheme wins
```

```
88              return s
89          # 2. an anchored LOCAL path is always local, never joined. Detect this OS-
INDEPENDENTLY ?
90          #    os.path.isabs is platform- AND Python-version-dependent (on Windows + py3.13 a
POSIX-style
91          #    "/data/x" is NOT absolute). Anchored == a leading "/" or "\\", or a Windows
drive prefix "X:".
92          if s.startswith(("/", "\\")) or (len(s) >= 2 and s[1] == ":" and s[0].isalpha()):
93              return s
94          if input_root:                    # 3. join a relative name under the configured
root
95              return input_root.rstrip("/") + "/" + s.lstrip("/")
96          if input_backend and input_backend != "local":   # 4. prefix a bucket/key with the
declared backend
97              return f"{input_backend}://{s}"
98          return s                          # 5. bare/relative local path ? unchanged
99
```

7. user (2026-06-21T09:17:14.181Z)

```
1       """Path-safety helpers for the API boundary (untrusted request fields)."""
2       import os
3       from typing import Any, Optional
4
5
6       def output_base(config: Any) -> str:
7           """The configured base dir for a segmenter's LOCAL scratch/staging artifacts
8           (COMPUTE_DECOUPLED_SERVING M3). Returns ``output.output_dir`` (default ``"output"``)
so the
9           chunk-staging / AI-handoff / temp dirs follow ``--output-dir`` /
``output.output_dir`` instead of
10          a hardcoded ``./output``. The DEFAULT is ``"output"`` ? byte-identical to the prior
literal.
11
12          This is the LOCAL output dir, deliberately NOT the storage ``workspace_root`` (which
may be a
13          ``gs://`` URI you can't ``makedirs``). Accepts the typed AppConfig or the legacy
dict-shim
14          (``.get("output", {}).get("output_dir")``); any miss or empty value falls back to
``"output"``."""
15          try:
16              out = config.get("output", {}).get("output_dir")
17          except Exception:
18              out = None
19          return out or "output"
20
21
22      def safe_output_filename(name: str) -> str:
23          """Validate an output filename is a BARE name (no dir part, no traversal, not
absolute).
24
25          `os.path.join(output_dir, name)` silently discards `output_dir` when `name` is
absolute,
26          and `..` segments escape it ? so an unchecked `output_filename` is an arbitrary-
write
27          primitive. Returns the name unchanged when safe; raises ValueError otherwise.
28          """
29          if not name or not name.strip():
30              raise ValueError("output_filename must not be empty")
31          if "\x00" in name:
32              raise ValueError("output_filename must not contain a null byte")
33          if "/" in name or "\\" in name:
34              raise ValueError(f"output_filename must not contain a path separator: {name!r}")
35          if os.path.isabs(name) or name != os.path.basename(name) or name in (".", ".."):
36              raise ValueError(f"output_filename must be a bare filename, got: {name!r}")
```

```
        return name


    def resolve_under_root(path: str, root: str) -> str:
        """Return the real path, asserting it is contained under `root` (raises ValueError
if not)."""
        rp = os.path.realpath(path)
        rr = os.path.realpath(root)
        try:
            contained = os.path.commonpath([rp, rr]) == rr
        except ValueError:  # different drives on Windows
            contained = False
        if not contained:
            raise ValueError(f"path {path!r} escapes the allowed root {root!r}")
        return rp


    def validate_input_path(path: str, allowed_root: Optional[str] = None) -> None:
        """Reject null bytes and (when an allowed_root is configured) out-of-root inputs.

        With no allowed_root (default, single-user local mode) only the null-byte check
applies,
        so the local 'point it at any of my videos' workflow is preserved. Setting an
        allowed_root (a future hosted/tenant deployment) enforces containment.
        """
        if "\x00" in path:
            raise ValueError("video_path must not contain a null byte")
        if allowed_root:
            resolve_under_root(path, allowed_root)

```

8. user (2026-06-21T09:17:14.451Z)

```
    """Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
P3).

    Extracted verbatim from backend.main to shrink the orchestrator; behavior is unchanged.
    The worker DRAINS the jobs table via `claim_due_job` and honours WAIT_AND_RESUME: a job
    that raises `JobWaiting` is parked (status 'waiting' + `next_poll_at` + progress) and
    re-claimed when due ? with no central scheduler (the DB is the clock).

    The monkeypatch contract these functions rely on (and that tests depend on):
    every cross-function seam ? `db_instance`, `global_config`, `_execute_processing`,
    `_arm_wake_timer`, `_get_executor`, `_MAX_DRAIN_WAKE_SEC` ? is resolved THROUGH the
    `backend.main` module object at call-time (`import backend.main as _main`), never as a
    bare local. `backend.main` re-exports these names, so `monkeypatch.setattr(backend.main,
    ...)` still intercepts the inter-function calls (e.g. a parked job arms the *patched*
    `_arm_wake_timer`, so no real daemon timer leaks in tests). Importing `backend.main`
lazily
    inside the functions also avoids the circular import at module load (main.py imports
this
    module; this module must not import main at import time).
    """
    import logging
    import threading
    import traceback
    from concurrent.futures import ThreadPoolExecutor
    from typing import Any, Dict, Optional

    logger = logging.getLogger(__name__)

    # Async job executor (DEFERRED_HARDENING_PLAN #16). max_workers=1 on purpose: a single
    # self-hosted box serializes GPU work, and the Gemini provider is rate-fragile under
    # parallel uploads (2026-06-10 ops learning: serial uploads + backoff). The segmenters
    # already parallelize their AI handoff internally via their own pools.
```

```
    _job_executor: Optional[ThreadPoolExecutor] = None


    def _get_executor() -> ThreadPoolExecutor:
        """Lazy module-global executor; tests monkeypatch this for inline execution."""
        global _job_executor
        if _job_executor is None:
            _job_executor = ThreadPoolExecutor(max_workers=1,
thread_name_prefix="jobworker")
        return _job_executor


    class JobWaiting(Exception):
        """Raised inside the pipeline to SELF-SCHEDULE a job (EXECUTION_POLICY.md P3,
        WAIT_AND_RESUME). It means "I can't finish now ? park me and resume in `delay_sec`",
        NOT a failure. The worker persists `next_poll_at` + `progress` (via
`set_job_waiting`),
        releases the thread, and re-claims the job when due (`claim_due_job`). No work to
date
        is the segmenter's `_execute_processing`, run UNDER the destroy-after-confirm
contract.

        The wiring is additive: the production segmenter path never raises this today (zero
        behaviour change), so a job runs exactly as before. The hook is what a later slice
        (resumable AI handoff) raises once a long op exhausts its in-process wait budget.
        """

        def __init__(self, delay_sec: float, progress: Optional[str] = None) -> None:
            super().__init__(f"job parked for {delay_sec:.0f}s")
            self.delay_sec = max(0.0, float(delay_sec))
            self.progress = progress


    def _job_to_request(job: Dict[str, Any]):
        """Reconstruct the submit-time ProcessRequest from a claimed job row, so a job
        re-claimed by `claim_due_job` (queued OR resumed-from-waiting) runs identically to
        the original submit. The row stores exactly the ProcessRequest fields."""
        import backend.main as _main
        return _main.ProcessRequest(
            video_path=job["video_path"],
            player_pool=job.get("player_pool"),
            max_frames=job.get("max_frames"),
            segmenter=job.get("segmenter"),
        )


    def _run_claimed_job(job: Dict[str, Any]) -> None:
        """Run ONE job already claimed (`status='running'`) by `claim_due_job`, recording
its
        terminal state ? OR parking it for resume on `JobWaiting` (WAIT_AND_RESUME).

        Job `status` is the EXECUTION state (queued|waiting|running|done|failed): 'done'
means
        the worker finished ? the pipeline's own verdict (validation failure etc.) lives in
        result["status"]; 'failed' means the worker itself crashed/raised; 'waiting' means
the
        job self-scheduled and will be re-claimed when `next_poll_at` is due.
        """
        import backend.main as _main
        db_instance = _main.db_instance
        job_id = job["id"]
        req = _main._job_to_request(job)
        try:
            result = _main._execute_processing(
                req, progress=lambda stage: db_instance.set_job_progress(job_id, stage))
```

```
            if result.get("video_id"):
                db_instance.set_job_video_id(job_id, result["video_id"])
            db_instance.mark_job_done(job_id, result)
        except _main.JobWaiting as w:
            # Cooperative re-enqueue: persist progress + next_poll_at, release the worker.
            logger.info(f"Job {job_id} parked for {w.delay_sec:.0f}s (resume when due).")
            db_instance.set_job_waiting(job_id, w.delay_sec, progress=w.progress)
        except Exception as e:
            logger.error(f"Job {job_id} crashed: {e}\n{traceback.format_exc()}")
            db_instance.mark_job_failed(job_id, f"{type(e).__name__}: {e}")


    def _drain_due_jobs() -> int:
        """One worker tick: claim and run every currently-runnable job (queued, or waiting whose
        `next_poll_at` is due), earliest-due first, until the table has nothing runnable.

        Each parked job (`JobWaiting` ? status 'waiting') drops out of this drain and is picked
        up by a FUTURE drain once due; we schedule that follow-up drain via `_schedule_drain` so
        a single-worker box keeps making progress with no central timer (the DB is the clock).
        Returns the number of jobs that reached a terminal/parked state this tick.
        """
        import backend.main as _main
        ran = 0
        while True:
            job = _main.db_instance.claim_due_job()
            if job is None:
                break
            ran += 1
            _main._run_claimed_job(job)
        # If anything is parked-but-not-yet-due, arrange a wake-up so it resumes without a new
        # submit. On the single-box executor this is a delayed re-drain; multi-worker just relies
        # on the next submit/drain. Best-effort ? never raises into the worker.
        _main._schedule_next_drain_if_waiting()
        return ran


    def _schedule_next_drain_if_waiting() -> None:
        """If any job is parked in 'waiting', arm a re-drain for when the soonest one is due, so a
        self-scheduled job resumes with no further client submit. The drain itself re-checks
        due-ness, so an early wake is harmless and a clamped wake for a far-future job is fine."""
        import backend.main as _main
        try:
            delay = _main.db_instance.seconds_until_next_due()
        except Exception as e:  # pragma: no cover - defensive; never wedge the worker
            logger.error(f"Could not compute next-due delay: {e}")
            return
        if delay is None:
            return  # nothing waiting
        # Clamp so a far-future job doesn't pin a thread forever; the drain re-checks due-ness.
        _main._arm_wake_timer(max(0.0, min(delay, _main._MAX_DRAIN_WAKE_SEC)))


    def _arm_wake_timer(delay_sec: float) -> None:
        """Arm a single daemon timer that re-submits a drain after ``delay_sec`` (no central
        scheduler ? the DB is the clock). Isolated as the sole timer-creating seam so tests can
```

```
143         monkeypatch it to a no-op and assert the *scheduling decision* without real
waiting."""
144         import backend.main as _main
145         timer = threading.Timer(delay_sec, lambda:
_main._get_executor().submit(_main._drain_due_jobs))
146         timer.daemon = True
147         timer.start()
148
149
150     #: A parked job further out than this is re-checked by a capped wake rather than one
long
151     #: sleeping timer ? keeps the wake cadence bounded without a central scheduler.
152     _MAX_DRAIN_WAKE_SEC = 60.0
153
```

9. user (2026-06-21T09:17:15.002Z)

```
1       """Google Cloud Storage backend ? first-class object buckets.
2
3       Uses ``google-cloud-storage``, imported lazily so ``import backend.storage`` never pulls
it;
4       install with the ``storage-gcs`` extra. Credentials come from Application Default
Credentials
5       (``GOOGLE_APPLICATION_CREDENTIALS`` pointing at a service-account JSON, or workload
identity) ?
6       never from config. A ``client`` may be injected for tests.
7       """
8       from __future__ import annotations
9
10      from datetime import timedelta
11      from typing import Any, Iterator, Optional
12
13      from backend.storage.base import StorageBackend
14      from backend.storage.ref import StorageRef, StorageStat
15
16
17      class GCSStorage(StorageBackend):
18          def __init__(self, *, client: Any = None, project: Optional[str] = None,
19                       signed_url_ttl: int = 3600, **_ignored) -> None:
20              self._ttl = signed_url_ttl
21              if client is not None:
22                  self._client = client
23                  return
24              from google.cloud import storage  # type: ignore  # lazy: only when a gcs:// URI
is used
25              self._client = storage.Client(project=project)
26
27          def _blob(self, ref: StorageRef):
28              return self._client.bucket(ref.bucket).blob(ref.key)
29
30          def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
31              if dst is None:
32                  raise ValueError("GCSStorage.fetch_to_local requires a dst path")
33              import os
34              dst = str(dst)
35              if os.path.exists(dst) and not overwrite:
36                  return dst
37              os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
38              self._blob(ref).download_to_filename(dst)
39              return dst
40
41          def put(self, local_path, ref, *, content_type=None) -> StorageRef:
42              self._blob(ref).upload_from_filename(str(local_path), content_type=content_type)
43              return ref
44
```

```
45          def exists(self, ref) -> bool:
46              return bool(self._blob(ref).exists())
47
48          def stat(self, ref) -> StorageStat:
49              b = self._blob(ref)
50              b.reload()
51              return StorageStat(size=int(b.size or 0), modified=b.updated, etag=b.etag,
52                                 content_type=b.content_type)
53
54          def list(self, prefix) -> Iterator[StorageRef]:
55              for blob in self._client.list_blobs(prefix.bucket, prefix=prefix.key):
56                  yield StorageRef("gcs", prefix.bucket, blob.name,
f"gcs://{prefix.bucket}/{blob.name}")
57
58          def delete(self, ref) -> None:
59              self._blob(ref).delete()
60
61          def signed_url(self, ref, *, ttl=3600, method="GET") -> str:
62              return self._blob(ref).generate_signed_url(
63                  version="v4", expiration=timedelta(seconds=ttl or self._ttl), method=method)
64
```

10. assistant (2026-06-21T09:17:19.790Z)

Let me verify the config defaults for `storage` (input_backend) and `security` (allowed_video_root), plus the DB write methods `add_video` and how segments persist.

11. user (2026-06-21T09:17:21.070Z)

```
feat/serving-m4-device-context
44c9f01 docs(serving): mark §7 tracker M4 ? + changelog (PR #283)
```

12. user (2026-06-21T09:17:21.423Z)

```
backend\config\__init__.py
backend\config\presets.py
backend\config\loader.py
backend\config\models.py
```

13. user (2026-06-21T09:17:22.336Z)

```
backend\config\presets.py:41:# these bundles ? ``input_backend`` (M2) and the configurable
output/workspace base (M3,
backend\config\models.py:280:class SecurityConfig(BaseModel):
backend\config\models.py:283:    allowed_video_root: Optional[str] = None
backend\config\models.py:287:    # arrive with PR-3 identity scoping. ? In hosted mode, set
allowed_video_root to
backend\config\models.py:297:class StorageConfig(BaseModel):
backend\config\models.py:310:    # input name is resolved against ``input_root`` (e.g.
``input_root="gs://bucket/videos"`` + "x.mp4"
backend\config\models.py:311:    # => ``gs://bucket/videos/x.mp4``) or, if ``input_root`` is
empty, prefixed with ``input_backend``
backend\config\models.py:315:    input_backend: Literal["local", "s3", "gcs", "gdrive"] =
"local"
backend\config\models.py:316:    input_root: str = ""
```

14. assistant (2026-06-21T09:17:24.545Z)

Let me read the config models to confirm defaults, and the DB `add_video` / segment persistence methods.

15. user (2026-06-21T09:17:25.555Z)

```
279
280     class SecurityConfig(BaseModel):
281         # When set, /api/process video_path must resolve under this directory (hosted/tenant
282         # mode). Default None preserves the single-user local 'any local file' workflow.
283         allowed_video_root: Optional[str] = None
284
285         # --- Chunked upload (B2, PR-2) ---
286         # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
287         # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
288         # contain upload_dir or /api/process will reject the uploaded paths.
289         upload_dir: str = "output/uploads"
290         # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
291         # free-tier edge proxies cap request bodies there (HOSTING_PLAN §2).
292         max_upload_mb: int = 4096
293         max_part_mb: int = 95
294         allowed_upload_extensions: List[str] = ["mp4", "mov"]
295
296
297     class StorageConfig(BaseModel):
298         """Pluggable storage backend (docs/DECOUPLED_COMPUTE/). Default ``local`` = today's
299         behaviour (filesystem paths, byte-for-byte). ``s3`` covers AWS + Cloudflare R2 + DO
Spaces
300         + MinIO (via ``endpoint_url``); ``gcs`` = Google Cloud Storage; ``gdrive`` = a
locally MOUNTED
301         Google Drive (``gdrive.mount_root``). Per-backend settings are loose dicts passed to
the constructor ?
302         **secrets are never stored here**, only endpoints / regions / file paths / source-
selectors
303         (boto3/google auth chains supply credentials). See ``backend/storage/``."""
304         backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
305         # Output root for produced artifacts. "" => fall back to output.output_dir (non-
breaking).
306         output_root: str = ""
307         # --- Input side (COMPUTE_DECOUPLED_SERVING M2; parallel to the output
`backend`/`output_root`).
308         # Default ``local`` / "" keeps today's behaviour: a bare path / ``local://`` URI is
read IN PLACE
309         # (identity, no copy). Set these to read inputs from a bucket / mounted Drive ? a
bare/relative
310         # input name is resolved against ``input_root`` (e.g.
``input_root="gs://bucket/videos"`` + "x.mp4"
311         # => ``gs://bucket/videos/x.mp4``) or, if ``input_root`` is empty, prefixed with
``input_backend``
312         # (``"gcs"`` + "bucket/x.mp4" => ``gcs://bucket/x.mp4``). An explicit scheme on the
input
313         # (``gs://?``/``s3://?``) or an absolute local path ALWAYS wins. Non-local inputs
are fetched to a
314         # scratch dir before processing (see ``backend.storage.acquire_local_input``).
315         input_backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
316         input_root: str = ""
317         # --- Per-video workspace (docs/PER_VIDEO_WORKSPACE.md) ---
318         # One folder per video keyed by the MD5 video_id:
<root>/[<workspace_prefix>/]<video_id>/...
319         # so a local deploy and a cloud (bucket) deploy share ONE relative layout (only the
root
320         # differs). DEFAULT-OFF: convergence is phased + flipped via a Launch Report (D5).
When OFF
321         # the legacy flat layout is used unchanged.
322         single_folder_per_video: bool = False
323         # Durable workspace root URI. "" => precedence: output_root, then output.output_dir.
324         workspace_root: str = ""
325         # Cloud namespace prefix under the root so per-video folders sit tidily beside
326         # videos/ labels/ weights/ (e.g. gs://bucket/workspaces/<video_id>/). "" = root
directly.
327         workspace_prefix: str = "workspaces"
```

```
328         local: dict[str, Any] = Field(default_factory=dict)
329         s3: dict[str, Any] = Field(default_factory=dict)
330         gcs: dict[str, Any] = Field(default_factory=dict)
331         gdrive: dict[str, Any] = Field(default_factory=dict)
332
333
334     class DeploymentConfig(BaseModel):
335         """Deployment profile ? one switch that selects a coherent bundle of compute +
storage
336         knobs (COMPUTE_DECOUPLED_SERVING M1, ``docs/COMPUTE_DECOUPLED_SERVING/README.md``
§4.3).
337
```

16. user (2026-06-21T09:17:26.968Z)

```
backend\infrastructure\database.py:287:    def add_video(self, video_id: str, filepath: str,
status: str, validation_results: List[Dict[str, Any]]) -> bool:
backend\infrastructure\database.py:341:    def add_play_segment(self, video_id: str, start_time:
float, end_time: float, server: Optional[str] = None, receiver: Optional[str] = None, metadata:
Optional[Dict[str, Any]] = None) -> Optional[int]:
backend\infrastructure\database.py:418:    def get_video(self, video_id: str) ->
Optional[Dict[str, Any]]:
backend\infrastructure\database.py:427:    def query_play_segments(self, video_id: str, filters:
Dict[str, Any]) -> List[Dict[str, Any]]:
backend\infrastructure\database.py:486:    def create_job(self, job_id: str, video_path: str,
segmenter: Optional[str] = None,
backend\infrastructure\database.py:514:    def set_job_video_id(self, job_id: str, video_id:
str) -> bool:
backend\infrastructure\database.py:520:    def mark_job_done(self, job_id: str, result:
Dict[str, Any]) -> bool:
backend\infrastructure\database.py:588:    def claim_due_job(self) -> Optional[Dict[str, Any]]:
```

17. assistant (2026-06-21T09:17:32.852Z)

Let me read the DB methods to confirm `add_video` is an unconditional upsert, that `create_job` stores the raw `video_path`, and that `claim_due_job` returns the stored `video_path`. Also confirm how segments are written (the segmenter writes to `play_segments`).

18. user (2026-06-21T09:17:33.103Z)

```
287         def add_video(self, video_id: str, filepath: str, status: str, validation_results:
List[Dict[str, Any]]) -> bool:
288             """Register or update a video row WITHOUT touching its child segments.
289
290             Uses UPSERT, not INSERT OR REPLACE: REPLACE deletes the conflicting row, which ?
291             with foreign_keys ON ? cascade-deletes every
play_segment/event/candidate_segment.
292             That made a status update (e.g. marking 'failed' on re-validation, or
'processed'
293             before the segmenter runs) silently destroy prior good results. UPSERT mutates
the
294             row in place; clearing segments is now an explicit, deliberate operation
295             (clear_video_data / prune_segments).
296             """
297             return self._execute_write(
298                 "INSERT INTO videos (id, filepath, status, validation_results) VALUES (?, ?,
?, ?) "
299                 "ON CONFLICT(id) DO UPDATE SET "
300                 "filepath=excluded.filepath, status=excluded.status, "
301                 "validation_results=excluded.validation_results",
302                 (video_id, filepath, status, json.dumps(validation_results)),
303                 "Failed to add video",
304             )
305
```

```
        def segment_watermarks(self, video_id: str) -> Tuple[int, int]:
            """Return (max play_segment id, max candidate_segment id) for a video (0 if
none).

            Captured before a (re)segmentation so the run's new rows (id > watermark) can be
            told apart from prior rows (id <= watermark), enabling destroy-AFTER-confirm.
            """
            with self._connect() as conn:
                cur = conn.cursor()
                p = cur.execute("SELECT COALESCE(MAX(id), 0) FROM play_segments WHERE
video_id = ?",
                                (video_id,)).fetchone()[0]
                c = cur.execute("SELECT COALESCE(MAX(id), 0) FROM candidate_segments WHERE
video_id = ?",
                                (video_id,)).fetchone()[0]
                return int(p), int(c)

        def prune_segments(self, video_id: str, *, play_upto: Optional[int] = None,
                           play_after: Optional[int] = None, cand_upto: Optional[int] =
None,
                           cand_after: Optional[int] = None) -> None:
            """Delete play/candidate segments by id range (events cascade from
play_segments).

            Used to finish a (re)segmentation: keep the NEW rows and drop the OLD ones on a
            successful run (``*_upto`` = the pre-run watermark), or drop the NEW partial
rows
            and keep the OLD ones when the run failed/produced nothing (``*_after``).
            """
            with self._connect() as conn:
                cur = conn.cursor()
                if play_upto is not None:
                    cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id <= ?",
(video_id, play_upto))
                if play_after is not None:
                    cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id > ?",
(video_id, play_after))
                if cand_upto is not None:
                    cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id <=
?", (video_id, cand_upto))
                if cand_after is not None:
                    cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id >
?", (video_id, cand_after))
                conn.commit()

        def add_play_segment(self, video_id: str, start_time: float, end_time: float,
server: Optional[str] = None, receiver: Optional[str] = None, metadata: Optional[Dict[str, Any]]
= None) -> Optional[int]:
            try:
                with self._connect() as conn:
                    cursor = conn.cursor()
                    cursor.execute(
                        "INSERT INTO play_segments (video_id, start_time, end_time,
duration, server, receiver, metadata) VALUES (?, ?, ?, ?, ?, ?, ?)",
                        (video_id, start_time, end_time, end_time - start_time, server,
receiver, json.dumps(metadata or {}))
                    )
                    conn.commit()
                    return cursor.lastrowid
            except Exception as e:
                logger.error(f"Failed to add play segment: {e}")
                return None

        def add_event(self, segment_id: int, timestamp: float, event_type: str, player:
Optional[str] = None, details: Optional[Dict[str, Any]] = None) -> bool:
```

```
356             return self._execute_write(
357                 "INSERT INTO events (segment_id, timestamp, type, player, details) VALUES
(?, ?, ?, ?, ?)",
358                 (segment_id, timestamp, event_type, player, json.dumps(details or {})),
359                 "Failed to add event",
360             )
361
362         def add_candidate_segment(self, video_id: str, detector: str, start_time: float,
end_time: float,
363                                   chunk_index: Optional[int] = None, confidence:
Optional[float] = None,
364                                   stats: Optional[Dict[str, Any]] = None,
365                                   detection_params: Optional[Dict[str, Any]] = None) ->
Optional[int]:
366             """Persist one raw (pre-confirmation) detection. `stats`/`detection_params` are
367             detector-specific dicts stored as JSON. Returns the new candidate id, or
None."""
368             try:
369                 with self._connect() as conn:
370                     cursor = conn.cursor()
371                     cursor.execute(
372                         """INSERT INTO candidate_segments
373                            (video_id, detector, chunk_index, start_time, end_time, duration,
374                             confidence, stats, detection_params)
375                            VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)""",
376                         (video_id, detector, chunk_index, start_time, end_time, end_time -
start_time,
377                          confidence, json.dumps(stats or {}), json.dumps(detection_params or
{}))
378                     )
379                     conn.commit()
380                     return cursor.lastrowid
381             except Exception as e:
382                 logger.error(f"Failed to add candidate segment: {e}")
383                 return None
384
385         def link_candidate_to_segment(self, candidate_id: int, segment_id: int) -> bool:
386             """Record that a candidate detection was confirmed as a given play_segment."""
```

19. user (2026-06-21T09:17:33.199Z)

```
486         def create_job(self, job_id: str, video_path: str, segmenter: Optional[str] = None,
487                        player_pool: Optional[List[str]] = None,
488                        max_frames: Optional[int] = None,
489                        policy: Optional[Dict[str, Any]] = None) -> bool:
490             """Insert a new job in state 'queued'. ``policy`` is an optional JSON
ExecutionPolicy
491             (EXECUTION_POLICY.md) the worker honours; None ? the worker's default policy."""
492             return self._execute_write(
493                 "INSERT INTO jobs (id, video_path, segmenter, player_pool, max_frames,
status, policy) "
494                 "VALUES (?, ?, ?, ?, ?, 'queued', ?)",
495                 (job_id, video_path, segmenter,
496                  json.dumps(player_pool) if player_pool is not None else None,
497                  max_frames, json.dumps(policy) if policy is not None else None),
498                 f"Failed to create job {job_id}",
499             )
500
501         def mark_job_running(self, job_id: str) -> bool:
502             return self._execute_write(
503                 "UPDATE jobs SET status='running', started_at=CURRENT_TIMESTAMP, "
504                 "progress='starting' WHERE id = ?", (job_id,),
505                 f"Failed to mark job {job_id} running",
506             )
507
```

```
        def set_job_progress(self, job_id: str, progress: str) -> bool:
            return self._execute_write(
                "UPDATE jobs SET progress=? WHERE id = ?", (progress, job_id),
                f"Failed to set progress for job {job_id}",
            )

        def set_job_video_id(self, job_id: str, video_id: str) -> bool:
            return self._execute_write(
                "UPDATE jobs SET video_id=? WHERE id = ?", (video_id, job_id),
                f"Failed to set video_id for job {job_id}",
            )

        def mark_job_done(self, job_id: str, result: Dict[str, Any]) -> bool:
            """Job EXECUTED to completion. The pipeline's own verdict (success/failed
            validation) is inside `result` ? job status 'done' means 'the worker
finished'."""
            return self._execute_write(
                "UPDATE jobs SET status='done', progress='finished', result=?, "
                "finished_at=CURRENT_TIMESTAMP WHERE id = ?",
                (json.dumps(result), job_id),
                f"Failed to mark job {job_id} done",
            )

        def mark_job_failed(self, job_id: str, error: str) -> bool:
            """Worker crashed / raised ? infrastructure failure, not a pipeline verdict."""
            return self._execute_write(
                "UPDATE jobs SET status='failed', error=?, "
                "finished_at=CURRENT_TIMESTAMP WHERE id = ?", (error, job_id),
                f"Failed to mark job {job_id} failed",
            )

        def get_job(self, job_id: str) -> Optional[Dict[str, Any]]:
            """Fetch one job; `result`/`player_pool` JSON deserialized."""
            with self._connect() as conn:
                row = conn.cursor().execute("SELECT * FROM jobs WHERE id = ?",
(job_id,)).fetchone()
                if not row:
                    return None
                d = dict(row)
                d["result"] = json.loads(d["result"]) if d["result"] else None
                d["player_pool"] = json.loads(d["player_pool"]) if d["player_pool"] else
None
                d["policy"] = json.loads(d["policy"]) if d.get("policy") else None
                return d

        # --- Self-scheduling execution policy (EXECUTION_POLICY.md P2)
-------------------------
        def set_job_policy(self, job_id: str, policy: Optional[Dict[str, Any]]) -> bool:
            """Attach/replace a job's JSON ExecutionPolicy."""
            return self._execute_write(
                "UPDATE jobs SET policy=? WHERE id = ?",
                (json.dumps(policy) if policy is not None else None, job_id),
                f"Failed to set policy for job {job_id}",
            )

        def set_job_waiting(self, job_id: str, delay_sec: float, progress: Optional[str] =
None) -> bool:
            """Self-schedule: park a job in 'waiting' until ``delay_sec`` from now,
releasing the
            worker. `claim_due_job` re-picks it when due ? cooperative, no master.
``next_poll_at`` is
            computed in SQLite's own UTC format so it compares directly to
CURRENT_TIMESTAMP."""
            return self._execute_write(
                "UPDATE jobs SET status='waiting', "
```

```
565                 "next_poll_at=datetime('now', ?), "
566                 "progress=COALESCE(?, progress) WHERE id = ?",
567                 (f"+{max(0, int(delay_sec))} seconds", progress, job_id),
568                 f"Failed to set job {job_id} waiting",
569             )
570
571         def seconds_until_next_due(self) -> Optional[float]:
572             """Seconds until the soonest parked ('waiting') job becomes due, so the worker
can
573             schedule a wake-up (EXECUTION_POLICY.md P3 self-scheduling). Returns 0.0 if one
is
574             already due, or None if nothing is waiting. A NULL ``next_poll_at`` ? due now
(0.0)."""
575             try:
576                 with self._connect() as conn:
577                     row = conn.cursor().execute(
578                         "SELECT MIN(CASE WHEN next_poll_at IS NULL THEN 0 ELSE "
579                         "  (julianday(next_poll_at) - julianday('now')) * 86400.0 END) AS
secs "
580                         "FROM jobs WHERE status='waiting'").fetchone()
581                 if row is None or row["secs"] is None:
582                     return None
583                 return max(0.0, float(row["secs"]))
584             except Exception as e:
585                 logger.error(f"Failed to compute next-due delay: {e}")
586                 return None
587
588         def claim_due_job(self) -> Optional[Dict[str, Any]]:
589             """Atomically claim ONE runnable job ? 'queued', or 'waiting' whose
``next_poll_at`` is due ?
590             flipping it to 'running'. Compare-and-set on (id, status) so concurrent workers
never
591             double-claim (the DB row-claim is the only coordination needed; no master).
Earliest-due
592             first. Returns the claimed job dict, or None if nothing is runnable."""
593             try:
594                 with self._connect() as conn:
595                     cur = conn.cursor()
596                     row = cur.execute(
597                         "SELECT id, status FROM jobs WHERE status='queued' "
598                         "OR (status='waiting' AND (next_poll_at IS NULL OR next_poll_at <=
CURRENT_TIMESTAMP)) "
599                         "ORDER BY COALESCE(next_poll_at, created_at) ASC LIMIT
1").fetchone()
600                     if not row:
601                         return None
602                     cur.execute(
603                         "UPDATE jobs SET status='running', started_at=CURRENT_TIMESTAMP "
604                         "WHERE id=? AND status=?", (row["id"], row["status"]))
605                     conn.commit()
606                     if cur.rowcount != 1:
607                         return None                     # lost the race to a concurrent
worker
608                 return self.get_job(row["id"])
609             except Exception as e:
610                 logger.error(f"Failed to claim a due job: {e}")
611                 return None
612
613         def fail_stale_jobs(self, reason: str = "interrupted by server restart") -> int:
614             """Crash recovery: any job still queued/running from a previous process is dead
615             (the executor is in-process). Mark failed so clients see a terminal state, not a
```

20. assistant (2026-06-21T09:17:38.681Z)

The whole chain is now verified. `claim_due_job` calls `get_job` which does `SELECT *` ? so

`video_path` round-trips. Let me do a final confirmation that the segmenter writes to `play_segments` via `add_play_segment` (so the DB write is part of the standard pipeline, not gs://-specific), and double-check the `gs://` normalization path once more by re-reading the relevant lines I already have.

The gs?gcs normalization is confirmed in `ref.py:60-61` (parse_uri) and the explicit-scheme-wins in `resolve_input_uri:87-88`. Let me confirm the segmenter persists segments via the DB.

21. user (2026-06-21T09:17:39.926Z)

```
backend\pipeline\segmenters\base.py:24:    def process_video(self, video_id: str, video_path:
str, player_pool: List[str] = None, max_frames: int = None) -> "Result[List[Dict[str, Any]]]":
backend\pipeline\segmenters\motion.py:20:    def process_video(self, video_id: str, video_path:
str, player_pool: List[str] = None, max_frames: int = None) -> List[Dict[str, Any]]:
backend\pipeline\segmenters\motion.py:201:            segment_id =
self.db.add_play_segment(video_id, start, end, server, receiver, metadata)
backend\pipeline\segmenters\gemini.py:492:            segment_id =
self.db.add_play_segment(video_id, start, end, None, None, metadata)
backend\pipeline\segmenters\gemini.py:505:    def process_video(self, video_id: str, video_path:
str, player_pool: List[str] = None, max_frames: int = None) -> List[Dict[str, Any]]:
backend\pipeline\segmenters\yolo_hybrid.py:346:                segment_id =
self.db.add_play_segment(video_id, start, end, None, None, metadata)
backend\pipeline\segmenters\yolo_hybrid.py:366:    def process_video(self, video_id: str,
video_path: str, player_pool: List[str] = None, max_frames: int = None) -> List[Dict[str, Any]]:
backend\pipeline\segmenters\trajectory_hybrid.py:138:    def process_video(self, video_id: str,
video_path: str,  # type: ignore[override]
backend\pipeline\segmenters\trajectory_hybrid.py:367:                segment_id =
self.db.add_play_segment(video_id, start, end, None, None, metadata)
```

22. assistant (2026-06-21T09:17:43.612Z)

The whole chain is confirmed. The segmenters call `db.add_play_segment(video_id, ...)` which writes into `play_segments` keyed by the `video_id` (MD5 of the fetched local file). This is the standard pipeline write ? identical regardless of whether the source was local or gs://. The `query_play_segments(video_id, ...)` reads them back, so the SPA can load via `?video=<id>`.

Let me record the verified trace.

23. user (2026-06-21T09:18:22.493Z)

Structured output provided successfully

24. assistant (2026-06-21T09:18:33.522Z)

The claim holds. I traced the complete chain with file:line evidence and verified each link:

Submit (`backend/main.py:269-300`): Under default config (`allowed_video_root=None`, `input_backend="local"`), `validate_input_path` with `allowed_root=None` only checks for null bytes, so a `gs://` string passes. The submit-time existence 404 check is explicitly gated behind `input_ref.backend == "local"` (line 283), so a bucket URI is **not** stat'd at submit. The raw `gs://` string is persisted into the jobs table.

gs?gcs normalization + explicit-scheme-wins (`backend/storage/ref.py:60-61, 87-88`): `parse_uri` rewrites `gs`?`gcs`, and `resolve_input_uri` returns an explicit-scheme input unchanged regardless of `input_backend`/`input_root`.

Worker round-trip (`backend/api/job_worker.py:59-90`): `claim_due_job`?`get_job` (`SELECT *`) returns the stored `video_path`; `_job_to_request` rebuilds `ProcessRequest(video_path=<gs://...>)` verbatim; on success `set_job_video_id` + `mark_job_done` are called.

Fetch + pipeline (`backend/main.py:159-237`): `acquire_local_input` downloads the blob to scratch (`gcs.py:30-39` `download_to_filename`), `compute_video_id` hashes the fetched bytes.

Unconditional DB write (`backend/main.py:190`, `database.py:287-304`): `add_video` UPSERTs the `videos` row on validation pass; segmenters write `play_segments` via `add_play_segment` keyed by `video_id` (`motion.py:201`, etc.). `query_play_segments` reads them back and `GET /api/jobs/{id}` exposes `video_id`, closing the round-trip for `?video=<id>`.

No code gaps. The only prerequisites are environmental runtime deps ? `google-cloud-storage` (lazily imported) and ADC auth ? and a fetch failure surfaces as a loud job failure, not a silent drop. One honest note recorded: `videos.filepath` stores the original `gs://` URI (the scratch copy is ephemeral), which is by design but means a later `/api/compile` would re-need the source ? outside this claim's process?queryable-video_id scope.